

JAVA PROGRAMMING MASTERCLASS

繼承與多形

「用繼承消除重複，用多形擴展彈性」

[← 返回目錄](#)

Outline

- **繼承 (Inheritance)** — extends 語法、存取修飾符、繼承類型、final、Sealed Classes、Records
- **IS-A 與 HAS-A 關係** — instanceof、聚合、組合
- **Override 與 Overload** — Override 規則、super、@Override、Overload 對比
- **多形 (Polymorphism)** — 編譯時期 vs 執行時期、型別轉型、Pattern Matching
- **靜態 / 動態綁定** — Static Binding vs Dynamic Binding
- **巢狀類別 (Nested Classes)** — 內部類別、方法類別、匿名類別

繼承 Inheritance

為什麼需要繼承？

Animal、Dog、Bird 三個類別大量重複程式碼：

類別	共有屬性	共有方法	獨有方法
Animal	name	eat() , sleep()	—
Dog	name	eat() , sleep()	barking()
Bird	name	eat() , sleep()	flying()

💡 繼承的目的：讓子類別直接引用父類別的屬性與方法，消除重複程式碼

繼承的語法 – extends

使用 **extends** 關鍵字，子類別自動擁有父類別的所有屬性與方法：

```
1  class Dog extends Animal {  
2      public void barking() {  
3          System.out.println(name + " 汪汪叫");  
4      }  
5  }
```

💡 Dog 無需再定義 **name**、**eat()**、**sleep()**，繼承後自動擁有

繼承範例 – Animal 與 Dog

```
1 // Animal.java
2 class Animal {
3     protected String name;
4     public Animal(String name) { this.name = name; }
5     public void eat() { System.out.println(name + " 吃東西"); }
6     public void sleep() { System.out.println(name + " 睡覺"); }
7 }
```

```
1 // Dog.java
2 class Dog extends Animal {
3     public Dog(String name) { super(name); }
4     public void barking() { System.out.println(name + " 汪汪叫"); }
5 }
```


父類別建構方法的啟動順序

建立子類別物件時，父類別的建構方法會先自動被呼叫：

```
1  // Animal.java
2  class Animal {
3      protected String name;
4      public Animal(String name) {
5          this.name = name;
6          System.out.println("Animal 建構");
7      }
8      public void eat() { System.out.println(name + " 吃東西"); }
9      public void sleep() { System.out.println(name + " 睡覺"); }
10 }
```

父類別建構方法的啟動順序 – 執行結果

```
1  // Dog.java
2  class Dog extends Animal {
3      public Dog(String name) {
4          super(name);
5          System.out.println("Dog 建構");
6      }
7      public void barking() { System.out.println(name + " 汪汪叫"); }
8
9      public static void main(String[] args) {
10         new Dog("旺財");
11         // 輸出：
12         // Animal 建構
13         // Dog 建構
14     }
15 }
```


存取修飾符與繼承

修飾符	同一類別	同套件	子類別	其他
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
(無修飾符)	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

💡 建議父類別屬性用 `protected`：子類別可直接存取，外部類別無法直接修改

protected 屬性與 super() 範例

```
1 // Animal.java
2 class Animal {
3     protected String name;
4     public Animal(String name) { this.name = name; }
5     public void eat() { System.out.println(name + " 吃東西"); }
6     public void sleep() { System.out.println(name + " 睡覺"); }
7 }
```

```
1 // Dog.java
2 class Dog extends Animal {
3     public Dog(String name) {
4         super(name); // 呼叫父類別的建構方法
5     }
6     public void barking() { System.out.println(name + " 汪汪叫"); }
7 }
```


super 關鍵字用法

用途	語法	說明
呼叫父類別建構方法	super(參數)	必須放在建構方法第一行
呼叫父類別方法	super.方法名()	用於 Override 後仍需呼叫父類別版本
存取父類別屬性	super.屬性名	父子類別有同名屬性時使用

super 關鍵字用法 – 範例

```
1  class Dog extends Animal {  
2      String name = "狗";           // 與父類別同名屬性  
3      Dog(String n) {  
4          super(n);                 // ① 第一行呼叫父類別建構方法  
5      }  
6      @Override  
7      void sound() {  
8          super.sound();             // ② 呼叫父類別的 sound()  
9          System.out.println(super.name); // ③ 存取父類別的 name  
10     }  
11 }
```


繼承類型

類型	說明
單一繼承 (Single)	一個子類別繼承一個父類別
分層繼承 (Hierarchical)	多個子類別繼承同一個父類別
多層次繼承 (Multi-Level)	子類別再被其他類別繼承 ($A \rightarrow B \rightarrow C$)
多重繼承 (Multiple)	Java 不支援，可改用介面 (Interface)

💡 Java 不允許同時繼承 2 個以上的父類別，但介面可以繼承多個介面

final 修飾符與繼承

用途	說明
final class	類別不能被繼承
final 方法	子類別無法 Override 此方法
靜態綁定	final 方法在編譯期決定，效能較佳

```
1  final class MathUtil { }           // 無法被繼承
2  class Animal {
3      public final void breathe() { } // 子類別無法 Override
4  }
```

💡 Java 內建範例：**String**、**Integer** 等 Wrapper 類別都宣告為 **final class**

密封類別 (Sealed Classes)

JDK 17 正式功能，允許父類別精確控制哪些子類別可以繼承它。

關鍵字

說明

sealed

宣告此類別為密封類別

permits

指定允許繼承的子類別清單

```
1 // 只允許 Circle 和 Square 繼承 Shape
2 public sealed class Shape permits Circle, Square { }
3
4 final class Circle extends Shape { }
5 final class Square extends Shape { }
```

密封子類別的修飾符限制

密封類別的子類別**必須**明確宣告為以下三種狀態之一：

修飾符	說明
final	禁止再被繼承（斷絕後代）
sealed	繼續保持密封，並指定自己的 permits
non-sealed	解除密封，允許任何類別繼承（回歸傳統）

```
1 public non-sealed class Circle extends Shape { } // 任何人都能繼承 Circle
```


Sealed Classes 與 Pattern Matching

密封類別搭配 **switch** (JDK 17+)，編譯器會檢查窮舉性：

```
1 // 如果 Shape 是 sealed，編譯器知道只有 Circle 和 Square
2 return switch (shape) {
3     case Circle c → c.radius() * c.radius() * Math.PI;
4     case Square s → s.side() * s.side();
5     // 不需要 default 區塊！編譯器保證所有可能都已涵蓋
6 };
```

💡 當你新增一個 permits 子類別時，編譯器會提醒你更新所有相關的 **switch** 邏輯

紀錄類別 (Records) 簡介

JDK 16 引入，只需宣告欄位，編譯器自動產生 constructor、getter、`equals`、`hashCode`、`toString`：

```
1 // 傳統寫法需要數十行；record 一行搞定
2 record Person(String name, int age) { }
3
4 Person p = new Person("炭治郎", 15);
5 System.out.println(p.name()); // "炭治郎"
6 System.out.println(p.age());  // 15
7 System.out.println(p);        // Person[name=炭治郎, age=15]
```


Records 與繼承限制

規則	說明
隱含 final	Record 無法被繼承
固定父類別	Record 隱含繼承 <code>java.lang.Record</code> ，不能再 <code>extends</code> 其他類別
實作介面	Record 可以實作多個介面

```
1 record Point(int x, int y) { } // ✓ 合法
2 // class Sub extends Point { } // ✗ Record 不能被繼承
3 // record R extends Animal { } // ✗ Record 不能繼承其他類別
4 interface Drawable { }
5 record Circle(double r) implements Drawable { } // ✓ 可實作介面
```


IS-A 與 HAS-A 關係

IS-A 關係 – 繼承

IS-A 代表「是一種」，子類別物件**也是**父類別的一種。

用 `instanceof` 驗證：

```
1  class Fish extends Animal {}  
2  class Bird extends Animal {}  
3  class Eagle extends Bird {}
```

```
1  Eagle eagle = new Eagle();  
2  System.out.println(eagle instanceof Bird); // true  
3  System.out.println(eagle instanceof Animal); // true
```


HAS-A 關係 – 聚合 vs 組合

類型	關鍵字	說明
聚合 (Aggregation)	無 extends	類別 A 的屬性是類別 B 的物件 (A HAS A B)
組合 (Composition)	用 extends	將多個類別的共用屬性抽取到父類別再繼承

💡 兩者目的相同：減少重複程式碼，提高可維護性

HAS-A 聚合範例 (Aggregation)

```
1  class Speed {  
2      protected int speed;  
3      public int getSpeed() { return speed; }  
4  }
```

```
1  class Car {  
2      private Speed s = new Speed(); // Car HAS A Speed  
3      public int getCarSpeed() { return s.getSpeed(); }  
4  }
```


HAS-A 組合範例 (Composition)

多個類別的共用屬性抽取到 **BasinInfo**，再繼承：

```
1  class BasinInfo {  
2      protected String id;  
3      protected String name;  
4  }
```

```
1  class Employee extends BasinInfo { int salary; }  
2  class Customer extends BasinInfo { int balance; }
```


Override 與 Overload

Override 最簡範例

子類別「重新定義」父類別的方法：

```
1  class Animal {  
2      public void move() { System.out.println("Animal 移動"); }  
3  }
```

```
1  class Dog extends Animal {  
2      @Override  
3      public void move() { System.out.println("Dog 跑步"); }  
4  }
```


Override 基本規則

規則	說明
方法名稱	必須 相同
參數列表	必須 相同
回傳型態	必須 相同 (或子型態)
存取權限	只能 放寬 ，不能縮減
不可覆寫	static 、 final 、 private 方法

方法隱藏 (Method Hiding) vs Override

比較項目	Override	方法隱藏
適用對象	實例方法	<code>static</code> 方法
決定時機	執行時期 (動態綁定)	編譯時期 (靜態綁定)
呼叫依據	物件的實際型態	變數的宣告型態

```
1 class Animal { static void sound() { System.out.println("Animal"); } }
```

```
1 class Dog extends Animal { static void sound() { System.out.println("Dog"); } }
```

```
1 Animal a = new Dog();  
2 a.sound(); // 輸出: Animal (由宣告型態 Animal 決定)
```


協變回傳型態 (Covariant Return Type)

Override 時，子類別可以回傳比父類別更具體的子型態：

父類別方法	子類別 Override	合法？
<code>Animal produce()</code>	<code>Dog produce()</code>	✓ Dog IS-A Animal
<code>Object clone()</code>	<code>Dog clone()</code>	✓
<code>int get()</code>	<code>double get()</code>	✗ 基本型態不適用

```
1 class Animal { Animal produce() { return new Animal(); } }
```

```
1 class Dog extends Animal {  
2     @Override  
3     Dog produce() { return new Dog(); }  
4 }
```


super 在 Override 中的應用

子類別呼叫父類別被覆寫的方法，用 `super.方法名()`：

```
1  class Dog extends Animal {  
2      @Override  
3      public void move() {  
4          super.move();           // 執行父類別的 move()  
5          System.out.println("Dog 跑步");  
6      }  
7  }
```

@Override 注解

加上 `@Override` 讓編譯器驗證方法簽名是否正確：

```
1  class Dog extends Animal {  
2      @Override  
3      public void move() { System.out.println("Dog 跑步"); }  
4  
5      // @Override          ← 若方法名拼錯，編譯時立即報錯  
6      // public void mov() { }  
7  }
```

💡 強烈建議每次 Override 父類別方法時都加上 `@Override`

Overload – 多重定義

方法名稱相同但參數不同，屬於編譯時期多形：

```
1  class Animal {  
2      public void eat(String food) {  
3          System.out.println("吃 " + food);  
4      }  
5      public void eat(String food, int amount) {  
6          System.out.println("吃 " + amount + " 份 " + food);  
7      }  
8  }
```

💡 Overload 靠參數的個數、型態、順序區別；與 Override 不同，不需繼承關係

多形 Polymorphism

兩種多形

類型	決定時機	機制
編譯時期多形 (Compile Time)	編譯期間	方法多重定義 (Overload)
執行時期多形 (Runtime)	執行期間	方法重新定義 (Override) + 向上轉型

💡 執行時期多形的 3 要件：① 有繼承關係 ② 子類別 **Override** 父類別方法 ③ 父類別變數參考子類別物件

執行時期多形 – 概念

先看沒有多形時，對不同動物分別呼叫 `move()`：

```
1  Dog dog = new Dog();  
2  Bird bird = new Bird();  
3  dog.move();    // 需要知道每個子類別的型態  
4  bird.move();
```

💡 每新增一種動物，就要多寫一段呼叫 — 難以擴展

執行時期多形 – 準備工作

各子類別各自 Override `move()` 方法：

```
1  class Animal { public void move() { System.out.println("Animal 移動");  
} }
```

```
1  class Dog extends Animal {  
2      @Override  
3      public void move() { System.out.println("Dog 跑步"); }  
4  }
```

```
1  class Bird extends Animal {  
2      @Override  
3      public void move() { System.out.println("Bird 飛翔"); }  
4  }
```


執行時期多形 – 用法

以父類別型態統一接收不同子類別物件：

```
1  Animal a1 = new Dog();    // Upcasting
2  Animal a2 = new Bird();   // Upcasting
3  a1.move(); // Dog 跑步
4  a2.move(); // Bird 飛翔
```

💡 同一個 `move()` 呼叫，依物件實際型態執行不同行為 — 這就是多形

向上轉型 Upcasting

將子類別物件指定給父類別型態變數，自動轉型：

特性	說明
自動轉型	不需強制轉型語法
可呼叫方法	只有父類別定義的方法
實際執行	子類別 Override 後的版本

```
1  Animal a = new Dog(); // 自動 Upcasting，a 只認識 Animal 的方法
```


向下轉型 Downcasting

將父類別型態變數轉回子類別，需強制轉型：

```
1  Animal a = new Dog();  
2  Dog dog = (Dog) a;    // Downcasting – 恢復存取 Dog 專屬方法  
3  dog.barking();        // 可呼叫 Dog 的 barking()
```

⚠ 若物件實際型態不符，執行時拋出 **ClassCastException**。建議先用 **instanceof** 判斷再轉型

Pattern Matching for instanceof

JDK 16 引入了更簡潔的 **Pattern Matching**，將 **instanceof** 判斷與轉型合併。

方式

語法

傳統方式

```
if (a instanceof Dog) { Dog d = (Dog) a; ... }
```

Pattern Matching

```
if (a instanceof Dog d) { d.barking(); }
```

```
1  Object obj = "Hello Java";
2
3  // 判斷的同時宣告變數 s，若符合則自動轉型
4  if (obj instanceof String s) {
5      System.out.println(s.toLowerCase()); // 直接使用 s
6  }
```

💡 變數 **s** 的作用域僅限於 **if** 區塊內 (或邏輯符合的範圍內)

靜態 / 動態綁定

Static Binding vs Dynamic Binding

類型	時機	適用
靜態綁定 (Static Binding)	編譯時期 (compile time)	<code>static</code> 、 <code>final</code> 、 <code>private</code> 方法
動態綁定 (Dynamic Binding)	執行時期 (runtime)	Override 後的一般方法

```
1  Animal a = new Dog();
2  a.move(); // 動態綁定：執行時才確定呼叫 Dog.move()
```

💡 執行時期多形的底層就是動態綁定：JVM 根據物件的實際型態決定執行哪個方法

巢狀類別 Nested Classes

巢狀類別的種類

類型	說明	使用場景
一般內部類別 (Inner Class)	定義在外部類別內，可存取外部所有成員	資料封裝、輔助類別
方法內部類別 (Method-local)	定義在方法內，只有該方法可使用	極少使用
匿名內部類別 (Anonymous)	宣告同時建立物件，一次性使用	Override 介面或抽象方法

一般內部類別 – 宣告

```
1  class OuterClass {  
2      int x = 10;  
3      class InnerClass {  
4          void display() {  
5              System.out.println("x = " + x); // 直接存取外部屬性  
6          }  
7      }  
8  }
```

一般內部類別 – 建立物件

必須先建立外部類別物件，再建立內部類別物件：

```
1 OuterClass outer = new OuterClass();  
2 OuterClass.InnerClass inner = outer.new InnerClass();  
3 inner.display(); // x = 10
```

💡 內部類別可宣告為 **private**，限制只有外部類別能使用

方法內部類別 (Method-local Inner Class)

類別宣告在方法內，只有該方法可使用此類別：

```
1  class School {  
2      void showRoom() {  
3          class MathRoom {          // 只有 showRoom() 能使用  
4              int students = 40;  
5          }  
6          MathRoom m = new MathRoom();  
7          System.out.println("學生數：" + m.students);  
8      }  
9  }
```

匿名內部類別 – 最簡範例

宣告的同時直接建立物件並 Override 方法：

```
1  Animal myAnimal = new Animal() {  
2      @Override  
3      public void move() {  
4          System.out.println("特殊移動方式");  
5      }  
6  };  
7  myAnimal.move();
```


匿名內部類別 – 當作參數傳送

Java 允許把匿名類別物件直接作為參數傳入方法：

```
1  obj.showAnimal(new Animal() {  
2      @Override  
3      public void move() {  
4          System.out.println("移動中 ... ");  
5      }  
6  });
```

⚠ 雖然合法，但降低程式碼可讀性與維護性，不建議大量使用

練習

任務說明

1. 繼承練習：建立父類別 **Father**，子類別 **Son** 和 **Daughter**

- **Father** : `name(String)`、`walk()` 印出 `name is walking!!!`
- **Son** : Override `walk()` 印 `name is walking~~~`，加上 `playBall()`
- **Daughter** : Override `walk()` 印 `name is walking@@@`，加上 `shopping()`

2. 多形練習：建立 **Animal**、**Dog**、**Bird**

- 各自 Override `move()` 方法
- 用 `Animal[] animals = { new Dog(), new Bird() }` 搭配迴圈呼叫 `move()`

練習

解題提示

1. 繼承結構

- 使用 `protected String name` 讓子類別能直接存取父類別屬性
- 子類別建構方法用 `super(name)` 初始化父類別屬性

2. 多形陣列

- 先確認 `Dog` 和 `Bird` 各自有 `@Override` 的 `move()`
- 宣告 `Animal[] animals = { new Dog("旺財"), new Bird("小翠") }`
- 用 `for` 迴圈呼叫 `animals[i].move()` 觀察多形效果

Q & A

本章結束

繼承讓程式碼更精簡，多形讓設計更彈性